

**KWAME NKRUMAH UNIVERSITY OF SCIENCE AND TECHNOLOGY,  
KUMASI  
COLLEGE OF SCIENCE  
FACULTY OF PHYSICAL AND COMPUTATIONAL SCIENCES  
DEPARTMENT OF MATHEMATICS**

**COURSE: SCIENTIFIC COMPUTING II  
MID-SEMESTER PROJECT WORK  
PROJECT REPORT, PART ONE**



**OBED ADU-GYAMFI  
INDEX NUMBER: 8597421**

**August, 2025**

# Contents

|          |                                                                                                                      |           |
|----------|----------------------------------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction.</b>                                                                                                 | <b>1</b>  |
| 1.1      | Overview. . . . .                                                                                                    | 1         |
| 1.2      | Concepts. . . . .                                                                                                    | 1         |
| <b>2</b> | <b>Algorithms.</b>                                                                                                   | <b>3</b>  |
| 2.1      | Mathematical Representation of Vector Multiplication, Scaler Product and C Programming Implementation. . . . .       | 3         |
| 2.2      | Mathematical Representation of Matrix Vector Multiplication and A C Programming Implementation. . . . .              | 4         |
| 2.3      | Mathematical Representation of Matrix Matrix Multiplication And A C Programming Implementation. . . . .              | 6         |
| <b>3</b> | <b>Test and Results</b>                                                                                              | <b>8</b>  |
| <b>4</b> | <b>conclusion</b>                                                                                                    | <b>10</b> |
| <b>A</b> | <b>Implementation of Vector Vector Multiplication, Matrix Vector Multiplication and Matrix Matrix Multiplication</b> | <b>11</b> |

# Chapter 1

## Introduction.

### 1.1 Overview.

Efficient numerical computations form the foundation of scientific computing and high-performance applications. At their core, vector and matrix operations are essential building blocks for algorithms in physics, engineering, data science, and optimization. This project focuses on the implementation of serial algorithms in C for three fundamental operations: vector-vector multiplication, matrix-vector multiplication, and matrix-matrix multiplication.

The implementation was structured using modular programming principles, with separate source and header files for vector and matrix operations. Input sizes were controlled through a macro parameter, and data buffers were internally generated to ensure independence from external inputs. Additionally, the computations were timed to analyze the performance of each operation as the problem size increased. The results were recorded for problem sizes ranging from  $N = 1000$  up to  $N = 40000$ , and scalability was analyzed by examining the time-to-solution across increasing dimensions.

This work serves as the baseline for later parallelization with MPI and provides insight into the computational cost of these fundamental mathematical operations when performed serially.

### 1.2 Concepts.

**Dot product** – also known as the "scalar product", is a binary operation that takes two vectors and returns a scalar quantity. The dot product of two vectors can be defined as the product of the magnitudes of the two vectors and the cosine of the angle between the two vectors. Alternatively, it is defined as the product of the projection of the first vector onto the second vector and the magnitude of the second vector. In the context of this report, the algebraic definition of the dot product is employed.

**Matrix Vector Multiplication** is a fundamental operation in computational mathematics that is widely employed in scientific and engineering applications. This procedure includes multiplying a 2D matrix by a 1D vector, resulting in a new vector. The matrix-vector multiplication process can be thought of as a linear combination of the matrix's columns, with the vector's components determining the results given to each column.

**Matrix Matrix Multiplication** Matrix-matrix multiplication is a fundamental operation in linear algebra and numerical computing, widely employed in various scientific and engineering fields. This operation involves the multiplication of two matrices, resulting in a new matrix. To multiply two matrices, the number of columns in the first matrix must equal the number of rows in the second matrix.

# Chapter 2

## Algorithms.

### 2.1 Mathematical Representation of Vector Multiplication, Scaler Product and C Programming Implementation.

Given two vectors  $A = (a_1, a_2, a_3, \dots, a_n)$  and  $B = (b_1, b_2, b_3, \dots, b_n)$ , the dot product of the vectors  $A$  and  $B$  is defined algebraically as:

$$a \cdot b = \sum_{i=0}^n a_i b_i \quad (2.1)$$

---

**Algorithm 1** Scalar Product of Two Vectors.

---

**Input:** Vectors  $\mathbf{A} = (a_1, a_2, \dots, a_n)$ ,  $\mathbf{B} = (b_1, b_2, \dots, b_n)$

**Output:** Scalar product  $s = \mathbf{A} \cdot \mathbf{B}$

$s \leftarrow 0$  **for**  $i \leftarrow 1$  **to**  $n$  **do**

$s \leftarrow s + a_i \times b_i$

**end**

**return**  $s$

---

```
1  #include "vector.h"
2
3  /**
4   * vector_vector - A vector-vector multiplication function
5   *
6   * Description: This function is a serial code that multiplies two
7   *               vectors and returns a scaler
8   */
```

```

8  * @m: This parameter is the dimension of the first vector A
9  * @n: This parameter also represents the dimension of the second
    vector B
10 * @vecA: This is a pointer to the first vector A
11 * @vecB: This is also a pointer to the second vector B.
12 *
13 * Return: A double.
14 */
15
16 double vector_vector(int m, int n, double *vecA, double *vecB){
17     double results = 0;
18     int i;
19     start = MPI_Wtime();
20
21     for (i = 0; i <= m; i++){
22         results += vecA[i] * vecB[i];
23     }
24     end = MPI_Wtime();
25     t_used = (double *)malloc(sizeof(double));
26     if (t_used != NULL){
27         *t_used = ((double)(end - start));
28     }
29     return (results);
30 }

```

## 2.2 Mathematical Representation of Matrix Vector Multiplication and A C Programming Implementation.

Given a matrix  $A = \begin{pmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \cdots & a_{mn} \end{pmatrix}$ , and vector  $B = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_n \end{pmatrix}$ ,

Then the resulting vector from the matrix-vector multiplication is given as :

$$c_k = a_{1k}b_k + a_{2k}b_k + \cdots + a_{nk}b_n = \sum_{k=0}^n a_{ik}b_k \quad (2.2)$$

---

**Algorithm 2** Matrix Vector Multiplication.

---

**Input:** matrix  $\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \cdots & a_{mn} \end{pmatrix}$ ,  $\mathbf{B} = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_n \end{pmatrix}$

**Output:** product  $s = \mathbf{A} \cdot \mathbf{B}$

**for**  $i \leftarrow 1$  **to**  $n$  **do**

**for**  $j \leftarrow 1$  **to**  $n$  **do**

$s_i \leftarrow s + a_{ij} \times b_j$

**end**

**end**

**return**  $s$

---

```

1  #include "matrix.h"
2
3  /*
4   * matrix_vector - A function the performs a matrix vector
5   * multiplication
6   * Description: This function accepts a matrix and a vector
7   * multiplies the matrix and vector.
8   *
9   * @m: Size or Dimension of the matrix A
10  * @n: Size of Dimension of the Vector B
11  * @matA: A pointer to an mxm matrix, matrix A
12  * @vecA: A pointer to the vector A
13  * @matB: This is a pointer to the resulting mxm matrix after the
14  * matrix vector multiplication.
15  *
16  * Return: Returns Void
17  */
18
19  void matrix_vector(int m, int n, double *matA, double *vecA,
20  double *matB){
21      int i, j;
22      start = clock();
23
24      for(i = 0; i < m; i++){
25          matB[i] = 0;
26          for (j = 0; j < n; j++){
27              matB[i] += matA[i * n + j] * vecA[j];

```

```

25         }
26     }
27     end = clock();
28     t_used = (double *)malloc(sizeof(double));
29     if (t_used != NULL){
30         *t_used = ((double)(end - start))/CLOCKS_PER_SEC;
31     }
32 }
33 }

```

### 2.3 Mathematical Representation of Matrix Matrix Multiplication And A C Programming Implementation.

Given a matrix  $A = \begin{pmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \cdots & a_{mn} \end{pmatrix}$ , and matrix  $B = \begin{pmatrix} b_{11} & b_{12} & b_{13} & \cdots & b_{1n} \\ b_{21} & b_{22} & b_{23} & \cdots & b_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ b_{m1} & b_{m2} & b_{m3} & \cdots & b_{mn} \end{pmatrix}$ ,

$A \times B$  :

$$c_{ik} = a_{1k}b_{1k} + a_{2k}b_{1k} + \cdots + a_{nk}b_{nk} = \sum_{k=0}^n a_{ik}b_{ik} \quad (2.3)$$

---

**Algorithm 3** Matrix Vector Multiplication.

---

**Input:** matrix  $A = \begin{pmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \cdots & a_{mn} \end{pmatrix}$ ,  $B = \begin{pmatrix} b_{11} & b_{12} & b_{13} & \cdots & b_{1n} \\ b_{21} & b_{22} & b_{23} & \cdots & b_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ b_{m1} & b_{m2} & b_{m3} & \cdots & b_{mn} \end{pmatrix}$

**Output:** product  $s = A \cdot B$

for  $i \leftarrow 1$  to  $n$  do

    for  $j \leftarrow 1$  to  $n$  do

        for  $k \leftarrow 1$  to  $n$  do

$localSum \leftarrow localSum + A[i \times n + k] \times B[k \times n + j]$

        end

$s[i \times n + j] \leftarrow localSum$

    end

end

return  $s$

---



```

1  #include "matrix.h"
2  /**
3   * matrix_matrix - This function performs a matrix matrix
      multiplication.
4   *
5   * Description: The function performs a matrix matrix multiplication
      of an mxn square matrix.
6   *
7   * @m: Size or dimension of matrix A
8   * @n: Size or dimension of matrix B
9   * matA: A pointer to the matrix A
10  * matB: A pointer to the matrix B
11  * matC: A pointer to the matrix C and stores the matrix product of
      matrix A and matrix B.
12  *
13  * Return: void
14  */
15
16 void matrix_matrix(int m, int n, double *matA, double *matB, double
      *matC){
17     start = clock();
18     int i, j, k;
19     double sum;
20
21     for (i = 0; i < m; i++){
22         for (j = 0; j < n; j++){
23             sum = 0;
24             for (k = 0; k < n; k++){
25                 sum += (matA[i * n + k] * matB[k *
26                     n + j]);
27             }
28             matC[i * n + j] = sum;
29         }
30     }
31     end = clock();
32     t_used = (double *)malloc(sizeof(double));
33     if (t_used != NULL){
34         *t_used = ((double)(end - start))/CLOCKS_PER_SEC;
35     }

```

# Chapter 3

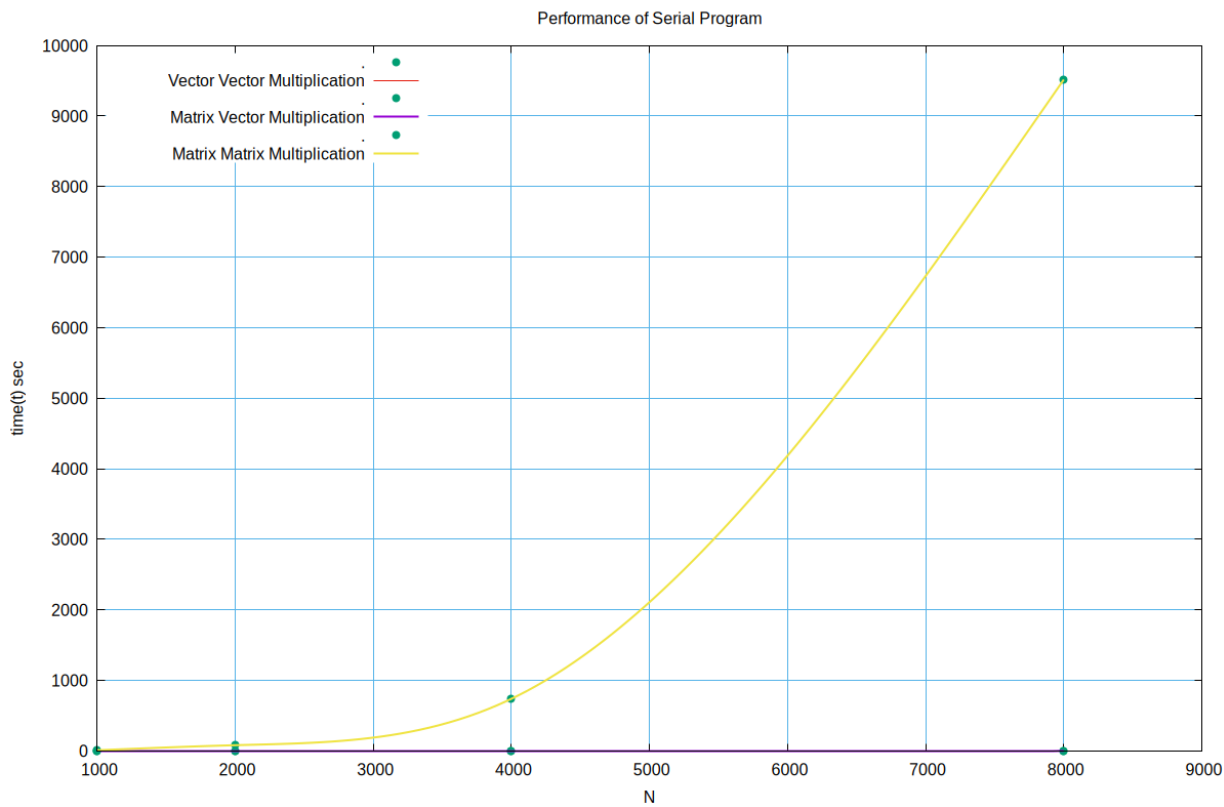
## Test and Results

Below is the execution records of the time taken by  $n$  core(s) from a serial run,  $n = 1$  core to  $n = 8$  cores processors with a vector size (N), number of cores, and execution time. Scalability tests performed with  $N = 2.5 \times 10^7$  for varying the number of cores.

Table 3.1: Time-To-Solution of  $N \times 2$

| N    | $V \cdot V/s$ | $M \cdot V/s$ | $M \cdot M/s$ |
|------|---------------|---------------|---------------|
| 1000 | 0.000004      | 0.004003      | 8.584200      |
| 2000 | 0.000007      | 0.015399      | 80.204949     |
| 4000 | 0.000013      | 0.063456      | 738.281960    |
| 8000 | 0.000025      | 0.255568      | 9516.001930   |

Figure 3.1: Time Vrs Number of Cores



# Chapter 4

## conclusion

The experimental results demonstrated the growth in computational cost as the matrix and vector dimensions increased, with the matrix–matrix multiplication operation being the most computationally intensive due to its cubic complexity. The scalability tests also highlighted how execution time grows rapidly with larger input sizes, providing a baseline for analyzing performance in parallel implementations.

This serial implementation provides a foundation for the subsequent parallelization using MPI. By first understanding the serial behavior and bottlenecks of these operations, it becomes possible to evaluate and compare the benefits of parallel computing in terms of speedup, efficiency, and scalability. Ultimately, this work contributes to appreciating the importance of optimizing basic linear algebra operations, as they form the core of numerous applications in scientific computing, engineering, and data analysis.

# Appendix A

## Implementation of Vector Vector Multiplication, Matrix Vector Multiplication and Matrix Matrix Multiplication

```
1  #include "8597421_vector.h"
2  #include "8597421_matrix.h"
3  /**
4   * Author: Obed Adu-Gyamfi
5   * Email: adugyamfiobed.tpp3@gmail.com
6   * github: @obedAdu-Gyamfi
7   */
8
9  /**
10 * main - Entry point
11 * Description: Executes and does functional calls
12 * Return: Always 0 on success
13 */
14 clock_t start;
15 clock_t end;
16 double *t_used;
17
18 int main(int argc, char **argv){
19     int i, j, k;
20     FILE *fd;
21     char *filename = "IndexNumber_serial.txt";
22     int m,n, size;
23     double *vecA;
```

## APPENDIX A. IMPLEMENTATION OF VECTOR VECTOR MULTIPLICATION, MATRIX VECTOR MU

```
24     double *vecB;
25     double *matV;
26
27     double *matB;
28     double *matA;
29     double vecans;
30 double *matC;
31     fd = fopen(filename, "w");
32     if (fd == NULL){
33         fprintf(stderr, "Unable to open %s", filename);
34         return (1);
35     }
36     fprintf(fd, "%-*s%-*s%-*s%-*s\n", SPACE, "N", SPACE, "V.V
37         /s", SPACE, "M.V/s", SPACE, "M.M/s");
38
39     if (argc != 2){
40         fprintf(stderr, "Usage <%s> arg\n", argv[0]);
41         exit(-1);
42     }
43     m = atoi(argv[1]);
44     for (k = 0; k < N; k++){
45         n = m;
46         size = m * n;
47         vecA = malloc(sizeof(double) * n);
48         vecB = malloc(sizeof(double) * n);
49         matV = malloc(sizeof(double) * n);
50
51         matA = malloc(sizeof(double) * size);
52         matB = malloc(sizeof(double) * size);
53         matC = malloc(sizeof(double) * size);
54
55         if (matA == NULL || matB == NULL){
56             fprintf(stderr, "Fialed to allocate memory
57                 for matrix!\n");
58             exit(-1);
59         }
60         for(i = 0; i < m; i++){
61             for (j = 0; j < n; j++){
62                 matA[i * n + j] = rand() / (double)
63                     RAND_MAX;
64                 matB[i * n + j] = rand() / (double)
65                     RAND_MAX;
66             }
67         }
68     }
```

## APPENDIX A. IMPLEMENTATION OF VECTOR VECTOR MULTIPLICATION, MATRIX VECTOR MU

```
63
64     if (vecA != NULL && vecB != NULL){
65         for (j = 0; j < n; j++){
66             vecA[j] = rand() /((double)RAND_MAX);
67             vecB[j] = rand() /((double)RAND_MAX;
68
69         }
70     }
71
72
73     vecans = vector_vector(m, n, vecA, vecB);
74     fprintf(fd, "%-*d_%-*f",SPACE, m, SPACE, *t_used);
75     free(t_used);
76
77     matrix_vector(m, n, matA, vecA, matV);
78     fprintf(fd, "%-*f", SPACE, *t_used);
79     free(t_used);
80     matrix_matrix(m, n, matA, matB, matC);
81     fprintf(fd, "%-*f\n", SPACE, *t_used);
82     free(t_used);
83
84     free(vecA);
85     free(vecB);
86     free(matA);
87     free(matB);
88     free(matC);
89     free(matV);
90     m *= 2;
91 }
92 fclose(fd);
93 return (0);
94 }
```